Documentación del proyecto

Carrito de compras

Proyecto de listas · Programación orientada a objetos en Python · Interfaz de consola

Alumno(s)	Rodrigo Quijano,Amir Hernandez,Emiliano Peniche,Marco Gomez
Materia	Estructura de Datos
Profesor(a)	**Lizbeth Hernández Olan**
Grupo	7
Fecha de entrega	20/09/2026

Fase 1. Contextualización

¿Qué se debe hacer?

Se debe desarrollar un programa de consola que simule el funcionamiento básico de un carrito de compras en una tienda. El sistema administra un catálogo de productos precargado y permite que el usuario seleccione productos y los agregue a un carrito. Toda la información se guarda de forma temporal, en memoria, utilizando **listas** como estructura de datos principal, y sobre ellas se aplican las operaciones de **inserción, búsqueda, actualización y eliminación** de elementos. En concreto, el programa debe:

- Contar con un **catálogo precargado** de productos; cada producto tiene ID, nombre y precio (el prototipo del proyecto añade además la categoría).
- Mostrar el catálogo completo y permitir **buscar** un producto por ID o por nombre.
- **Agregar** productos al carrito, seleccionando el producto e indicando la cantidad.
- **Visualizar** el carrito: productos agregados, cantidad por producto, precio unitario, subtotal por producto y total acumulado.
- **Modificar** la cantidad de un producto del carrito, actualizando el subtotal automáticamente.
- **Eliminar** productos del carrito mediante su ID o su nombre.
- **Vaciar** el carrito y reiniciar la compra.
- **Finalizar** la compra mostrando la lista de productos comprados, la cantidad total de artículos y el monto total a pagar.

¿Qué se necesita?

Necesidad	Detalle
Datos de entrada	Opción del menú, ID o nombre de un producto, cantidades y confirmaciones (s/n) escritas por el usuario.
Datos precargados	Catálogo inicial de 5 productos (tabla siguiente).
Estructuras de datos	Una **lista de productos** (catálogo) y una **lista del carrito**. Es obligatorio usar listas para almacenar la información.
Paradigma	Programación orientada a objetos: clases con atributos y métodos que representan las entidades del problema.
Procesos y cálculos	Subtotal por producto (precio × cantidad), total acumulado, total de artículos y total de productos distintos.
Validación y errores	Validar todas las entradas del usuario y manejar los errores básicos (producto inexistente, cantidad inválida, carrito vacío, etc.).
Interfaz	Consola (el modo gráfico es opcional y no se implementa).
Herramientas	Python 3.7 o superior (probado con Python 3.14), terminal y un editor de texto.

Catálogo inicial precargado:

ID	Producto	Categoría	Precio
1	Teclado	Accesorios	$450
2	Mouse	Accesorios	$250
3	Monitor	Electrónica	$3200
4	Audífonos	Audio	$800
5	Memoria USB	Almacenamiento	$180

¿Qué elementos se identifican?

Elemento	Tipo	Descripción
Usuario (cliente)	Actor	Persona que interactúa con el menú de la consola para comprar.
Producto	Entidad	Artículo de la tienda con ID, nombre, categoría y precio.
Catálogo	Colección	Lista de los productos disponibles para la compra.
Carrito	Colección	Lista de las líneas de compra del usuario; calcula total y total de artículos.
Ítem del carrito	Entidad	Línea del carrito: un producto y la cantidad que se desea comprar; calcula su subtotal.
Menú principal	Interfaz	Nueve opciones numeradas que dirigen cada operación.
Cantidad, subtotal, total, artículos	Datos calculados	Valores derivados que se recalculan automáticamente al cambiar el carrito.
Mensajes y validaciones	Control	Avisos de éxito, error o confirmación (s/n) mostrados al usuario.

Flujo general de datos:

Entradas	Proceso	Salidas
Opción de menú, ID o nombre del producto, cantidad, confirmación s/n	Búsqueda en la lista del catálogo; inserción, actualización y eliminación en la lista del carrito; cálculo de subtotales y totales	Tablas de catálogo, resultados y carrito; mensajes de confirmación o error; resumen final de compra

Fase 2. Definición de los requerimientos funcionales

Listado de los requerimientos del contexto planteado:

Requerimientos funcionales
R1 – Mostrar catálogo
R2 – Buscar producto (por ID o por nombre)
R3 – Agregar producto al carrito
R4 – Ver carrito
R5 – Modificar cantidad de un producto
R6 – Eliminar producto del carrito
R7 – Vaciar carrito
R8 – Finalizar compra
R9 – Menú principal y salir

Cada requerimiento funcional se documenta con el esquema descriptivo solicitado (nombre, descripción, entrada y salidas).

Nombre	R1 – Mostrar catálogo
Descripción	Presenta al usuario, en forma de tabla, todos los productos del catálogo precargado. Cada producto se muestra con su ID, nombre, categoría y precio para que pueda elegir qué comprar.
Entrada	Opción 1 del menú principal. Después, ENTER para volver al menú.
Salidas	Tabla «CATÁLOGO DE PRODUCTOS» con los 5 productos (ID, Producto, Categoría, Precio).

Nombre	R2 – Buscar producto
Descripción	Permite localizar productos del catálogo por **ID** (coincidencia exacta) o por **nombre** (coincidencia parcial, sin distinguir mayúsculas, minúsculas ni acentos). Si no hay coincidencias, avisa al usuario.
Entrada	Opción 2 del menú; tipo de búsqueda (1 = ID, 2 = nombre); ID (entero) o texto del nombre.
Salidas	Tabla «RESULTADOS DE LA BÚSQUEDA» con los productos encontrados, o el mensaje «No se encontraron productos.»

Nombre	R3 – Agregar producto al carrito
Descripción	El usuario selecciona un producto del catálogo (por ID o por nombre) e indica la cantidad. El sistema inserta una nueva línea en la lista del carrito; si el producto ya estaba, suma la cantidad a su línea. Valida que el producto exista y que la cantidad sea un entero mayor o igual a 1.
Entrada	Opción 3 del menú; ID o nombre del producto; cantidad (entero positivo).
Salidas	Nombre y precio unitario del producto y el mensaje «¡Producto agregado al carrito!» (con aviso cuando la cantidad se acumula sobre una línea existente).

Nombre	R4 – Ver carrito
Descripción	Muestra los productos agregados con su cantidad, precio unitario y subtotal (precio × cantidad), además del total acumulado de la compra. Si el carrito está vacío, lo informa.
Entrada	Opción 4 del menú.
Salidas	Tabla «CARRITO DE COMPRAS» (ID, Producto, Precio, Cant., Subtotal) y «TOTAL: $…»; o el mensaje «El carrito está vacío.»

Nombre	R5 – Modificar cantidad
Descripción	Permite seleccionar un producto del carrito y cambiar su cantidad. El programa recalcula de forma automática el subtotal de esa línea y el total del carrito.
Entrada	Opción 5 del menú; ID o nombre del producto del carrito; nueva cantidad (entero mayor o igual a 1).
Salidas	Producto y cantidad actual, mensaje «¡Cantidad actualizada!», nuevo subtotal y total del carrito.

Nombre	R6 – Eliminar producto del carrito
Descripción	Quita del carrito un producto identificado por su **ID** o por su **nombre**, pidiendo confirmación antes de eliminarlo.
Entrada	Opción 6 del menú; ID o nombre del producto del carrito; confirmación (s/n).
Salidas	«¡Producto eliminado del carrito!», o «Operación cancelada.» si el usuario responde «n».

Nombre	R7 – Vaciar carrito
Descripción	Elimina todos los productos del carrito y reinicia la compra, previa confirmación del usuario.
Entrada	Opción 7 del menú; confirmación (s/n).
Salidas	«¡Carrito vaciado!» (lista del carrito vacía), o «Operación cancelada.»

Nombre	R8 – Finalizar compra
Descripción	Cierra la compra mostrando la lista de productos comprados, la cantidad total de artículos y el monto total a pagar. Después reinicia el carrito para comenzar una nueva compra.
Entrada	Opción 8 del menú.
Salidas	«RESUMEN DE COMPRA» (ID, Producto, Cant., Subtotal), número de productos, total de artículos, «Total a pagar: $…» y «Gracias por su compra.»

Nombre	R9 – Menú principal y salir
Descripción	Muestra el menú con las 9 opciones, valida la opción elegida y repite hasta que el usuario decide salir. Al salir, la información en memoria se pierde.
Entrada	Número de opción del 1 al 9.
Salidas	Menú principal; aviso «Opción inválida…» cuando la entrada no es válida; mensaje de despedida al elegir la opción 9.

Requerimientos no funcionales

Id	Requerimiento	Cómo se cumple en el programa
RNF1	Interfaz basada en consola	Toda la interacción usa `input()`/`print()`; menú numerado y tablas de texto.
RNF2	Validación de entradas del usuario	Opciones de menú, cantidades (enteros ≥ 1), IDs o nombres y respuestas s/n se validan y se vuelven a pedir.
RNF3	Manejo de errores básicos	Excepción propia `CarritoError` para reglas de negocio; captura en el ciclo principal; cierre limpio ante Ctrl+C o fin de entrada.
RNF4	Paradigma orientado a objetos	Seis clases con atributos, métodos, propiedades, composición y una relación de herencia (`CarritoError`).
RNF5	Uso obligatorio de listas	`Catalogo.productos` y `Carrito.items` son listas de Python; todas las operaciones se hacen sobre ellas.

Fase 3. Diseño

A partir de los requerimientos se identificaron las clases del sistema, con sus atributos y métodos, y se construyó el diagrama de clases UML.

Clases identificadas

Clase	Responsabilidad
Producto	Representa un artículo del catálogo (ID, nombre, categoría y precio).
ItemCarrito	Línea del carrito: relaciona un producto con la cantidad deseada y calcula su subtotal.
Catalogo	Guarda en una lista los productos disponibles y permite buscarlos por ID o nombre.
Carrito	Guarda en una lista los ítems; agrega, modifica, elimina y vacía; calcula totales.
AplicacionCarrito	Interfaz de consola: muestra el menú, lee y valida las entradas y coordina catálogo y carrito.
CarritoError	Excepción propia (hereda de `Exception`) para las reglas de negocio violadas.

Diagrama de clases UML

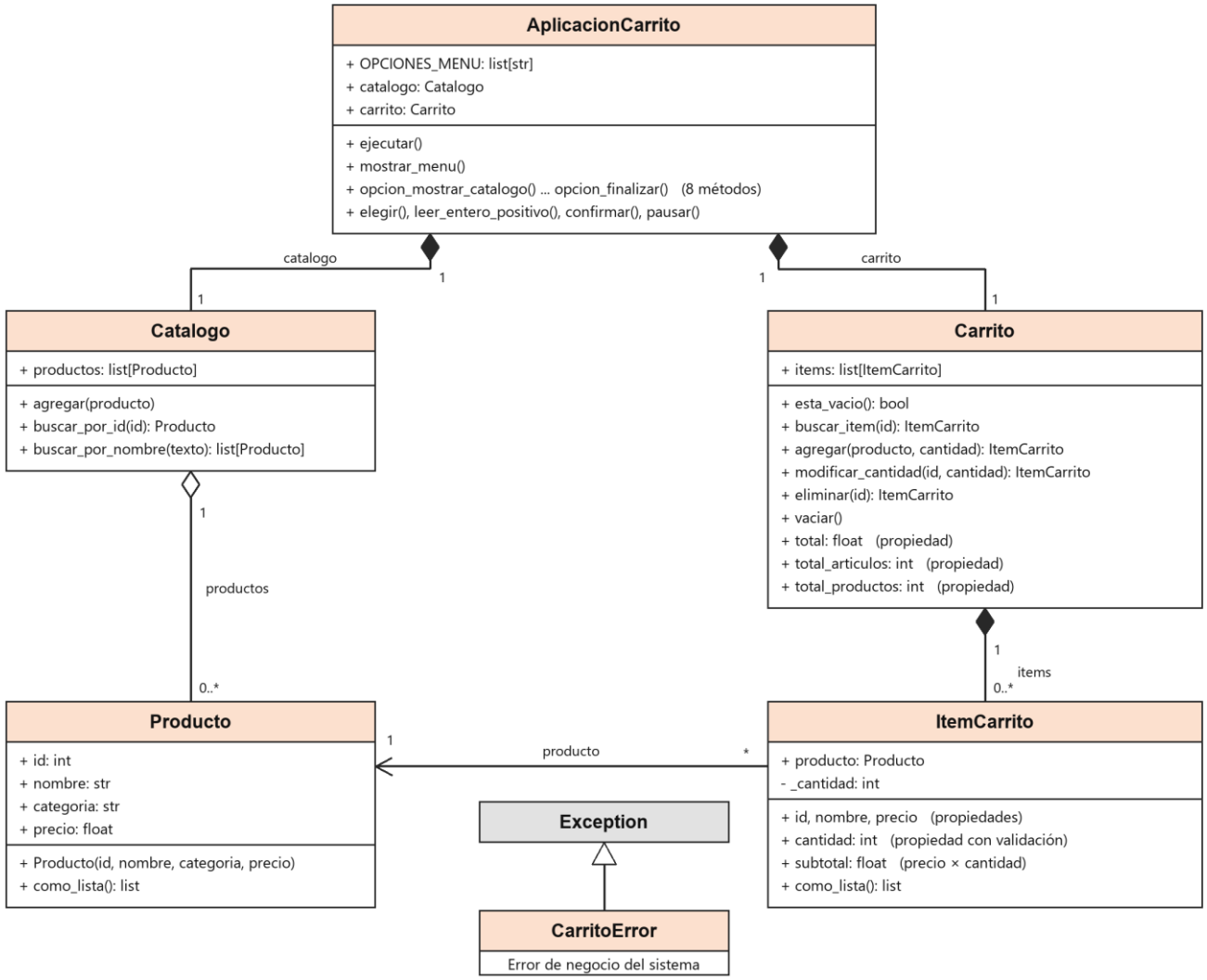


Figura 1. Diagrama de clases del carrito de compras (◆ composición, ◇ agregación, → asociación, △ herencia)

Relaciones entre clases

Relación	Tipo	Multiplicidad	Explicación
AplicacionCarrito → Catalogo	Composición	1 a 1	La aplicación crea el catálogo precargado al iniciar y es su dueña.
AplicacionCarrito → Carrito	Composición	1 a 1	La aplicación crea y controla el carrito de la compra actual.
Catalogo → Producto	Agregación	1 a 0..*	El catálogo contiene la lista `productos`.
Carrito → ItemCarrito	Composición	1 a 0..*	El carrito contiene la lista `items`; las líneas no existen fuera del carrito.
ItemCarrito → Producto	Asociación dirigida	* a 1	Cada línea del carrito hace referencia a un producto del catálogo.
CarritoError → Exception	Herencia	—	`CarritoError` especializa a `Exception`. Es la única relación de herencia del diseño.

Además, `Producto`, `ItemCarrito`, `Catalogo` y `Carrito` lanzan `CarritoError` cuando se viola una regla (dependencia), y `AplicacionCarrito` la captura para informar al usuario sin detener el programa.

Descripción de las clases (atributos y métodos)

Producto

Atributos
`+ id: int` — identificador único
`+ nombre: str`
`+ categoria: str`
`+ precio: int | float` — no puede ser negativo

Métodos `+ __init__(id_producto, nombre, categoria, precio)` — asigna los atributos y valida el precio `+ como_lista(): list` — devuelve [id, nombre, precio]

ItemCarrito
Atributos `+ producto: Producto` — producto de la línea `- _cantidad: int` — cantidad (se accede con la propiedad)
Métodos `+ __init__(producto, cantidad)` `+ id, nombre, precio` — propiedades de solo lectura que delegan en el producto `+ cantidad: int` — propiedad; el setter exige un entero ≥ 1 `+ subtotal: float` — propiedad: precio × cantidad `+ como_lista(): list` — devuelve [id, nombre, precio, cantidad]

Catalogo
Atributos + productos: list[Producto] — lista de productos disponibles
Métodos + __init__(productos=None) + agregar(producto) — inserta en la lista (rechaza IDs repetidos) + buscar_por_id(id_producto): Producto | None + buscar_por_nombre(texto): list[Producto] — coincidencia parcial sin acentos ni mayúsculas

Carrito
Atributos + items: list[ItemCarrito] — lista de líneas de compra
Métodos + __init__() + esta_vacio(): bool + buscar_item(id_producto): ItemCarrito | None + agregar(producto, cantidad): ItemCarrito — inserta o acumula la cantidad + modificar_cantidad(id_producto, nueva_cantidad): ItemCarrito + eliminar(id_producto): ItemCarrito + vaciar() + total: float — propiedad: suma de subtotales + total_articulos: int — propiedad: suma de cantidades + total_productos: int — propiedad: número de líneas

AplicacionCarrito
Atributos + OPCIONES_MENU: list[str] — atributo de clase con las 9 opciones + catalogo: Catalogo + carrito: Carrito
Métodos + __init__() — crea el catálogo precargado y un carrito vacío + ejecutar() — ciclo principal del menú + mostrar_menu() + opcion_mostrar_catalogo(), opcion_buscar(), opcion_agregar(), opcion_ver_carrito(), opcion_modificar(), opcion_eliminar(), opcion_vaciar(), opcion_finalizar() + imprimir_catalogo(productos, titulo), imprimir_carrito(titulo, con_precio), imprimir_lista_corta(elementos), titulo(texto, ancho) + elegir(elementos, mensaje) — pide ID o nombre y resuelve coincidencias + leer_entero_positivo(mensaje), confirmar(mensaje), pausar(mensaje), limpiar_pantalla(), carrito_vacio()

CarritoError *«hereda de Exception»*
Atributos *(sin atributos propios)*
Métodos *(sin métodos propios)*

Funciones de utilidad (fuera de las clases)

Función	Propósito
`normalizar(texto)`	Convierte a minúsculas y quita acentos para comparar nombres.
`formatear_dinero(monto)`	Da formato de moneda: 450 → «$450»; 450.5 → «$450.50».
`construir_tabla(encabezados, filas, alineacion)`	Devuelve las líneas de una tabla de texto con columnas alineadas.
`filtrar_por_referencia(elementos, referencia)`	Busca en una lista por ID (si es número) o por nombre.
`catalogo_inicial()`	Crea el catálogo precargado con los 5 productos.

Estructuras de datos: las listas

La información vive en dos listas de Python. `Catalogo.productos` contiene objetos `Producto` y `Carrito.items` contiene objetos `ItemCarrito`. El método `como_lista()` de ambas clases entrega cada elemento con la forma simple que muestra el prototipo del proyecto:

```
Lista de productos (catálogo)
[[1, "Teclado", 450], [2, "Mouse", 250], [3, "Monitor", 3200],
 [4, "Audífonos", 800], [5, "Memoria USB", 180]]

Lista del carrito (ejemplo con Teclado x3 y Monitor x1)
[[1, "Teclado", 450, 3], [3, "Monitor", 3200, 1]]
```

Las operaciones sobre las listas son: **inserción** (append al agregar), **búsqueda** (recorrido con `for`), **actualización** (cambio de la cantidad de un ítem) y **eliminación** (`remove` y `clear`).

Fase 4. Codificación

Los programas fuente se escribieron a partir del diagrama de clases de la fase anterior. Todo el código está en un solo archivo, `carrito_compras.py` (se incluye completo en el Anexo A).

Para ejecutarlo: abrir una terminal en la carpeta del archivo y escribir:

```
python carrito_compras.py
```

Implementación de los métodos (parte lógica)

Cada método del diagrama está implementado con la lógica que se resume a continuación. Ejemplo: el método `Carrito.agregar` inserta una nueva línea en la lista o acumula la cantidad si el producto ya estaba en el carrito:

```
def agregar(self, producto, cantidad):
    """Agrega el producto; si ya estaba en el carrito suma la cantidad."""
    item = self.buscar_item(producto.id)
    if item is None:
        item = ItemCarrito(producto, cantidad)
        self.items.append(item)
    else:
        item.cantidad = item.cantidad + cantidad
    return item
```

El método `Carrito.modificar_cantidad` busca la línea y le asigna la nueva cantidad; el subtotal se recalcula solo porque es una propiedad calculada:

```
def modificar_cantidad(self, id_producto, nueva_cantidad):
    item = self.buscar_item(id_producto)
    if item is None:
        raise CarritoError("Ese producto no está en el carrito.")
    item.cantidad = nueva_cantidad
    return item
```

La búsqueda por ID o por nombre (usada al agregar, modificar y eliminar) está en la función `filtrar_por_referencia`:

```
def filtrar_por_referencia(elementos, referencia):
    """Busca en una lista de elementos con .id y .nombre.

    Si la referencia es un número se busca por ID; si no, por nombre
    (coincidencia exacta primero y, si no hay, coincidencia parcial).
    """
    ref = referencia.strip()
    if not ref:
        return []
    if ref.isascii() and ref.isdigit():
        return [e for e in elementos if e.id == int(ref)]
    ref_norm = normalizar(ref)
    exactos = [e for e in elementos if normalizar(e.nombre) == ref_norm]
    if exactos:
        return exactos
    return [e for e in elementos if ref_norm in normalizar(e.nombre)]
```

Codificación de fórmulas

Fórmula	Código en el programa	Ubicación
Subtotal = precio × cantidad	`return self.precio * self.cantidad`	ItemCarrito.subtotal
Total = Σ subtotales	`return sum(item.subtotal for item in self.items)`	Carrito.total
Total de artículos = Σ cantidades	`return sum(item.cantidad for item in self.items)`	Carrito.total_articulos
Total de productos = número de líneas	`return len(self.items)`	Carrito.total_productos
Cantidad acumulada = cantidad actual + nueva	`item.cantidad = item.cantidad + cantidad`	Carrito.agregar

Asignaciones

Código	Propósito
`self.items = []`	Inicializa la lista del carrito vacía (`Carrito.__init__`).
`self.productos = list(productos) if productos else []`	Inicializa la lista del catálogo.
`self.catalogo = catalogo_inicial()`	La aplicación recibe el catálogo precargado.
`self._cantidad = valor`	Guarda la cantidad ya validada en el ítem.
`item = self.buscar_item(producto.id)`	Localiza la línea del producto en el carrito.
`item.cantidad = nueva_cantidad`	Actualiza la cantidad de una línea.

Condicionales y ciclos

Código	Propósito
`if not isinstance(valor, int) or valor < 1:`	Rechaza cantidades que no sean enteros ≥ 1.
`if precio < 0:`	Rechaza precios negativos al crear un producto.
`if item is None:`	Detecta que el producto no está en el carrito.
`if len(coincidencias) == 1:`	Acepta la selección cuando hay una sola coincidencia.
`if respuesta in ("s", "si"):`	Interpreta la confirmación del usuario.
`if self.carrito.esta_vacio():`	Avisa que el carrito está vacío antes de operar.
`if accion is None:`	Detecta una opción de menú inválida.
`while True:`	Repite el menú y las lecturas hasta recibir una entrada válida.
`for item in self.items:`	Recorre la lista del carrito para buscar un producto.

Ejemplo de validación de entradas (lectura de una cantidad):

```
@staticmethod
def leer_entero_positivo(mensaje):
    """Pide un entero >= 1. Devuelve None si el usuario deja la entrada vacía."""
    while True:
        texto = input(mensaje).strip()
        if not texto:
            return None
        if texto.isascii() and texto.isdigit() and int(texto) >= 1:
            return int(texto)
        print("Entrada inválida: escriba un número entero mayor o igual a 1 "
              "(ENTER para cancelar).")
```

Documentación del código

- Cada clase y los métodos y funciones clave tienen su **docstring** que explica su propósito.
- Se usan **nombres descriptivos** en español (`agregar`, `modificar_cantidad`, `total_articulos`…).
- Se incluyen **comentarios** en los puntos no evidentes (por ejemplo, el reinicio del carrito al finalizar).
- El módulo inicia con un docstring que describe el programa y su estructura de listas.

Fase 5. Pruebas

Se comprobó que el programa cumple con los requerimientos establecidos. Las pruebas se ejecutaron sobre el programa real, simulando las entradas del usuario y verificando automáticamente las salidas y el estado de las listas. Resultado global: **27 de 27 casos aprobados**.

Verificación de requerimientos

Requerimiento	Casos de prueba	Resultado
R1 – Mostrar catálogo	CP01	Cumple
R2 – Buscar producto	CP02, CP03, CP04, CP05, CP06	Cumple
R3 – Agregar producto al carrito	CP07, CP08, CP09, CP10, CP11, CP12	Cumple
R4 – Ver carrito	CP13, CP22	Cumple
R5 – Modificar cantidad	CP14, CP15, CP22	Cumple
R6 – Eliminar producto del carrito	CP16, CP17, CP18, CP22	Cumple
R7 – Vaciar carrito	CP19, CP20, CP22	Cumple
R8 – Finalizar compra	CP21, CP22	Cumple
R9 – Menú principal y salir	CP23, CP24	Cumple
RNF1 – Interfaz de consola	Todos los casos	Cumple
RNF2 – Validación de entradas	CP06, CP10, CP11, CP12, CP15, CP18, CP23	Cumple
RNF3 – Manejo de errores	CP25, CP26	Cumple
RNF4 – Orientación a objetos	CP26, CP27	Cumple
RNF5 – Uso de listas	CP27	Cumple

Casos de prueba

Id	Req.	Prueba y entrada	Resultado esperado	Resultado obtenido	Estado
CP01	R1	**Mostrar el catálogo completo** Opción 1	Tabla con los 5 productos (ID, producto, categoría y precio).	Se muestran los 5 productos con sus categorías y precios.	Aprobado
CP02	R2	**Buscar producto por ID existente** Opción 2 → 1 → ID 3	Se muestra únicamente el Monitor.	Resultado con una sola fila: Monitor, Electrónica, $3200.	Aprobado
CP03	R2	**Buscar producto por ID inexistente** Opción 2 → 1 → ID 99	Mensaje de que no se encontraron productos.	Se muestra «No se encontraron productos.».	Aprobado
CP04	R2	**Buscar por nombre sin acentos ni mayúsculas** Opción 2 → 2 → «audifonos»	Encuentra el producto «Audífonos».	Se muestra la fila 4 | Audífonos | Audio | $800.	Aprobado
CP05	R2	**Buscar por nombre parcial** Opción 2 → 2 → «usb»	Encuentra «Memoria USB».	Se muestra la fila 5 | Memoria USB | Almacenamiento | $180.	Aprobado

Id	Req.	Prueba y entrada	Resultado esperado	Resultado obtenido	Estado
CP06	R2	**Tipo de búsqueda inválido** Opción 2 → 7	Mensaje de opción inválida sin cerrar el programa.	Se muestra «Opción inválida. Elija 1 o 2.» y el programa continúa.	Aprobado
CP07	R3	**Agregar producto por ID** Opción 3 → ID 1 → cantidad 2	Producto agregado; Teclado x2 en el carrito (subtotal $900).	Mensaje «¡Producto agregado al carrito!»; carrito: [1, Teclado, 450, 2].	Aprobado
CP08	R3	**Agregar producto por nombre** Opción 3 → «mouse» → cantidad 1	Mouse x1 agregado al carrito.	Se identifica el Mouse por nombre y se agrega con cantidad 1.	Aprobado
CP09	R3	**Agregar un producto que ya está en el carrito** Teclado x2 y luego Teclado x1	No se duplica la línea: la cantidad pasa a 3.	Una sola línea con cantidad 3 y aviso «ahora lleva 3 unidades».	Aprobado
CP10	R3	**Producto inexistente al agregar** Opción 3 → ID 99 → ID 1 → cantidad 1	Avisa que no existe y vuelve a pedir el producto.	Se muestra el aviso para el ID 99, se solicita otro y se agrega el ID 1.	Aprobado
CP11	R3	**Cantidad inválida al agregar** Cantidad: «abc», 0, -2, 2.5 y finalmente 2	Rechaza las 4 entradas inválidas y acepta 2.	Cuatro avisos de entrada inválida; se agrega con cantidad 2.	Aprobado
CP12	R3	**Nombre ambiguo al elegir producto** Producto «o» (coincide con varios) → ID 1	Muestra las coincidencias y pide el ID.	Se listan los 5 productos y se solicita indicar el ID.	Aprobado
CP13	R4	**Ver carrito con productos** Teclado x2, Mouse x1 → opción 4	Tabla con cantidades, precios, subtotales ($900 y $250) y TOTAL $1150.	Se muestran ambas líneas y «TOTAL: $1150».	Aprobado
CP14	R5	**Modificar cantidad y recalcular subtotal** Teclado x2 → opción 5 → ID 1 → nueva cantidad 3	Cantidad 3, subtotal $1350 calculado automáticamente.	«¡Cantidad actualizada!» con «Nuevo subtotal: $1350».	Aprobado
CP15	R5	**Cantidad inválida al modificar** Nueva cantidad: 0, «x» y finalmente 4	Rechaza 0 y texto; acepta 4.	Dos avisos de entrada inválida; la cantidad queda en 4.	Aprobado
CP16	R6	**Eliminar producto por ID con confirmación** Teclado x2, Mouse x1 → opción 6 → ID 2 → «s»	El Mouse se elimina; el Teclado permanece.	«¡Producto eliminado del carrito!»; queda [1, Teclado, 450, 2].	Aprobado
CP17	R6	**Eliminar por nombre y cancelar** Opción 6 → «mouse» → «n»	Operación cancelada; el carrito no cambia.	«Operación cancelada.»; el carrito conserva 2 líneas.	Aprobado
CP18	R6	**Respuesta s/n inválida** Confirmación: «quizas» y luego «s»	Repite la pregunta hasta recibir s o n.	Se muestra «Responda 's' para sí o 'n' para no.» y luego elimina.	Aprobado
CP19	R7	**Vaciar el carrito confirmando** Teclado x2, Mouse x1 → opción 7 → «s»	El carrito queda vacío (total $0).	«¡Carrito vaciado!»; la lista del carrito queda vacía.	Aprobado
CP20	R7	**Vaciar el carrito cancelando** Opción 7 → «n»	Los productos permanecen en el carrito.	«Operación cancelada.»; el carrito conserva 2 líneas.	Aprobado

Id	Req.	Prueba y entrada	Resultado esperado	Resultado obtenido	Estado
CP21	R8	**Finalizar compra** Teclado x3 y Monitor x1 → opción 8	Resumen con 2 productos, 4 artículos y total a pagar $4550; el carrito se reinicia.	Resumen: Productos 2, Artículos 4, Total a pagar $4550; carrito vacío después.	Aprobado
CP22	R4-R8	**Operaciones con el carrito vacío** Opciones 4, 5, 6, 7 y 8 sin productos	Mensaje «El carrito está vacío.» en cada una.	Se muestra el aviso 5 veces y no ocurre ningún error.	Aprobado
CP23	R9	**Opción de menú inválida** Menú: «abc», 0 y 10	Mensaje de error y el menú se vuelve a mostrar.	Tres avisos «Opción inválida. Ingrese un número del 1 al 9.».	Aprobado
CP24	R9	**Salir del programa** Opción 9	Mensaje de despedida y fin del programa.	Se muestra «Gracias por usar el programa. ¡Hasta pronto!» y termina.	Aprobado
CP25	RNF3	**Cierre inesperado (Ctrl+C / fin de entrada)** Se interrumpe la entrada en el menú y en medio de una operación	El programa termina con mensaje de despedida, sin errores.	En ambos casos finaliza limpiamente con «Programa finalizado.».	Aprobado
CP26	RNF3	**Errores de la lógica de negocio** Cantidad 0 en ItemCarrito, precio negativo, ID duplicado y eliminar ID inexistente	Las 4 operaciones lanzan CarritoError.	Se lanzó CarritoError en las 4 situaciones.	Aprobado
CP27	RNF5	**Almacenamiento en listas** Se consulta la lista del catálogo y la del carrito (Teclado x3, Monitor x1)	Ambas estructuras son listas con los datos esperados.	Catálogo: lista de 5 productos; carrito: [[1, 'Teclado', 450, 3], [3, 'Monitor', 3200, 1]].	Aprobado

Documentos de entrega final

- Este documento: **Documentación del proyecto** (fases 1 a 5).
- **carrito_compras.py**: programa fuente (listado completo en el Anexo A).
- **diagrama_clases.png** y carpeta **evidencias/**: imágenes usadas en este documento.

Evidencias de ejecución (impresión de pantallas)

Las siguientes pantallas corresponden a una sesión real del programa, con las entradas indicadas. El texto en amarillo es lo que teclea el usuario. Salvo la primera, las pantallas omiten la repetición del menú principal para ahorrar espacio.



Figura 2. Menú principal y opción 1: mostrar catálogo

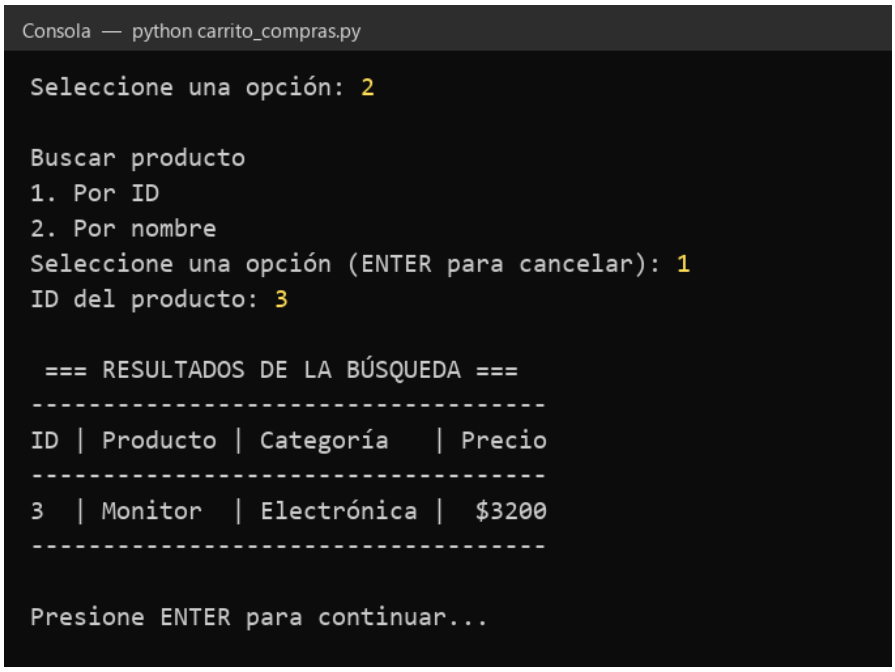


Figura 3. Opción 2: buscar producto por ID

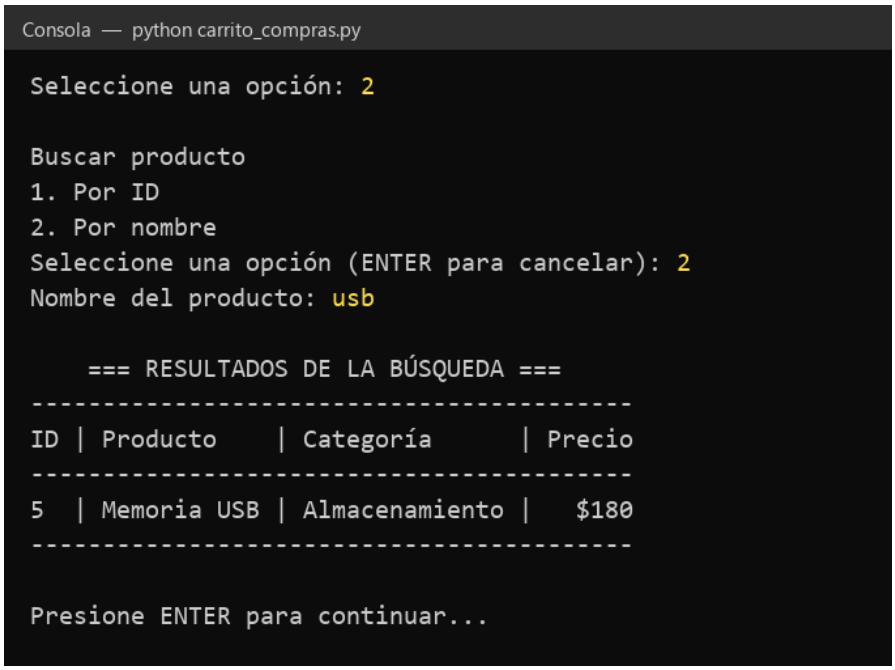


Figura 4. Opción 2: buscar producto por nombre

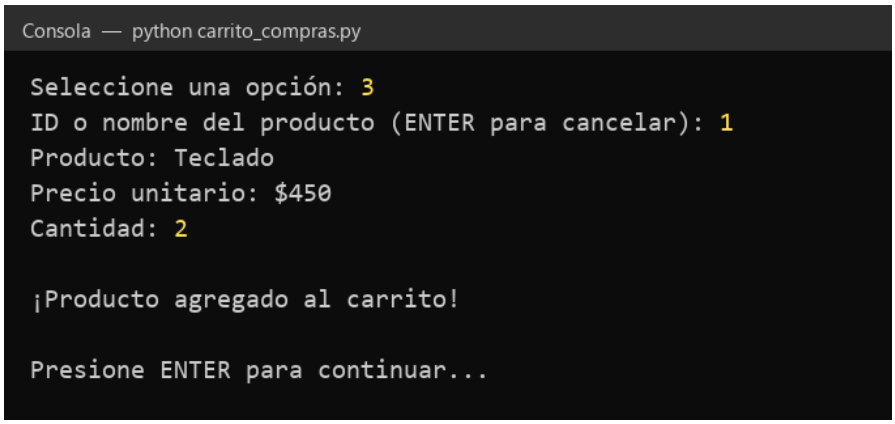


Figura 5. Opción 3: agregar producto al carrito (por ID)

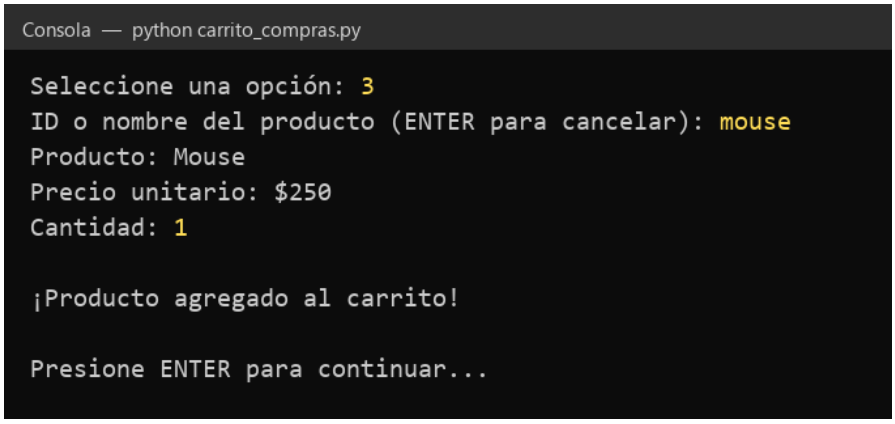


Figura 6. Opción 3: agregar producto al carrito (por nombre)

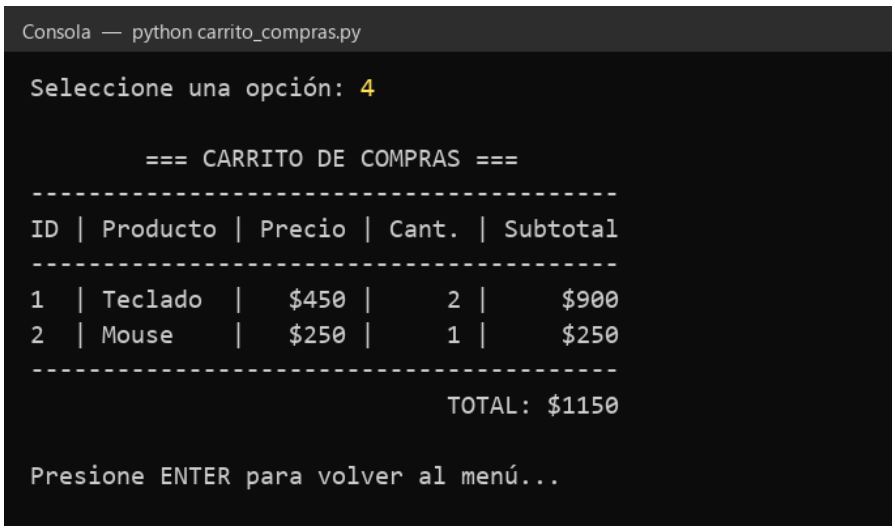


Figura 7. Opción 4: ver carrito

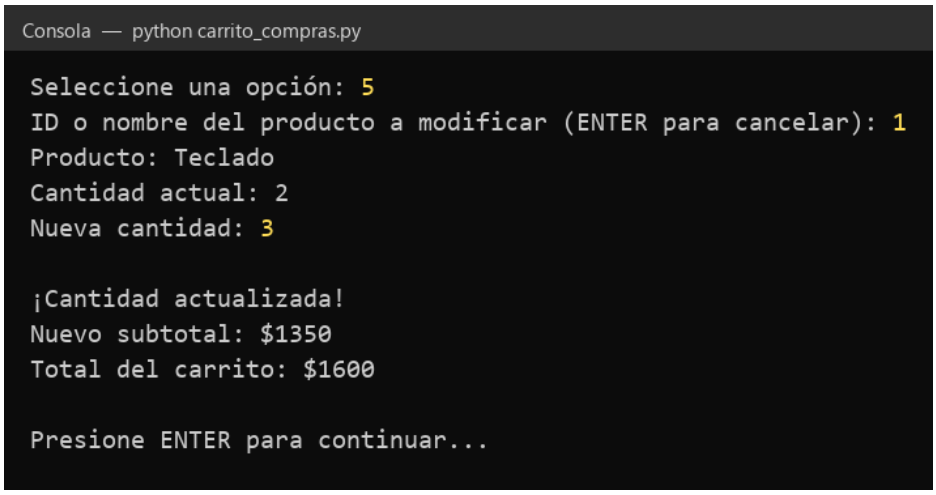


Figura 8. Opción 5: modificar cantidad

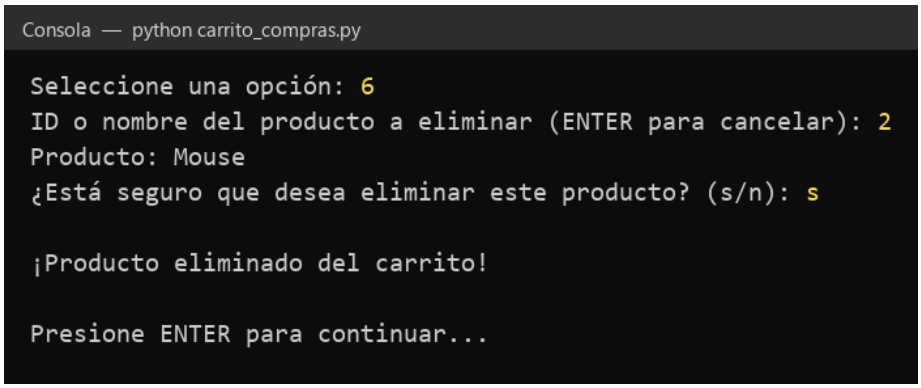


Figura 9. Opción 6: eliminar producto del carrito

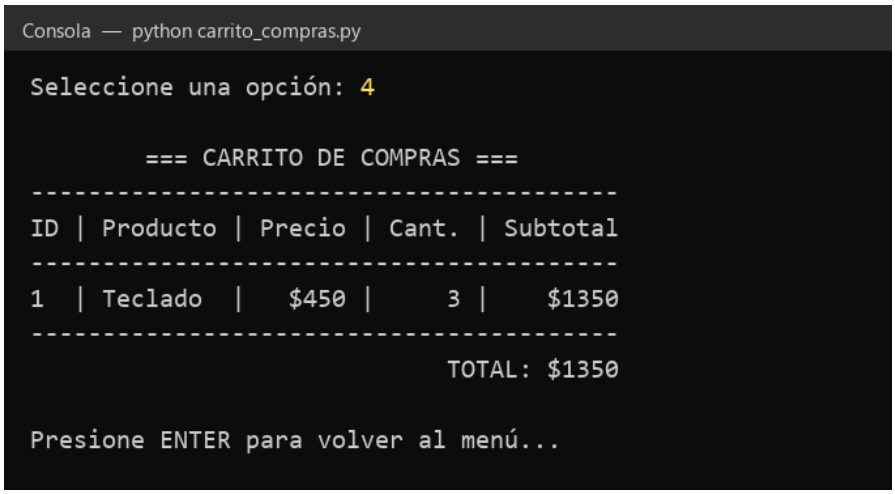


Figura 10. Opción 4: ver carrito actualizado

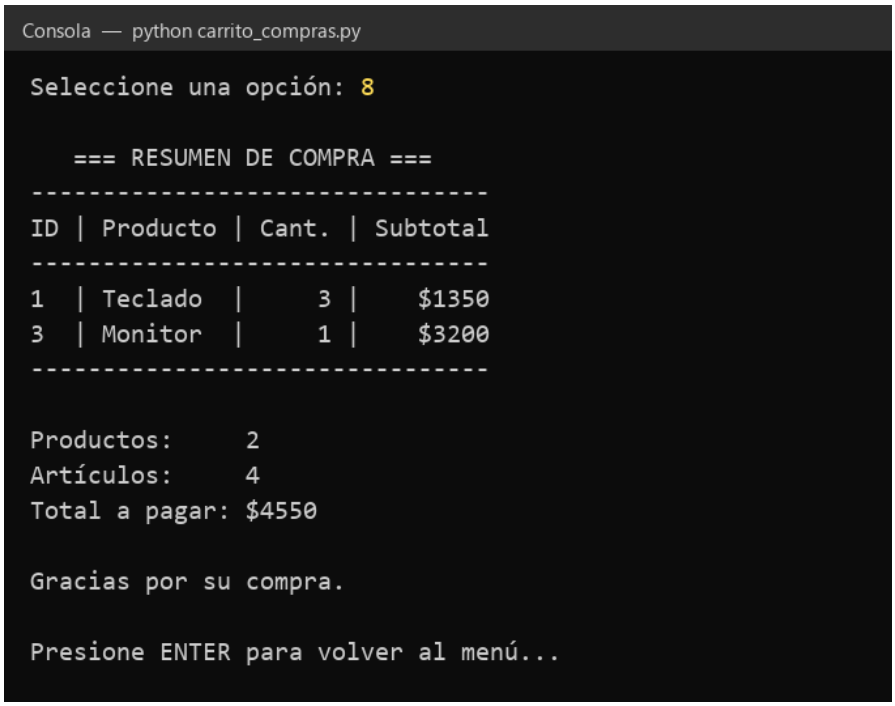


Figura 11. Opción 8: finalizar compra

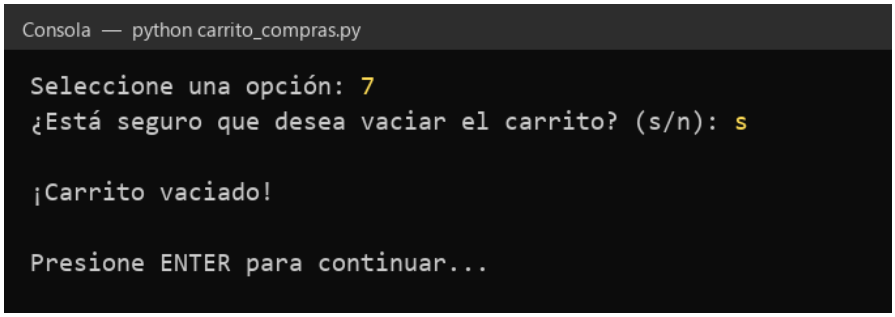


Figura 12. Opción 7: vaciar carrito



Figura 13. Carrito vacío: aviso al intentar ver el carrito

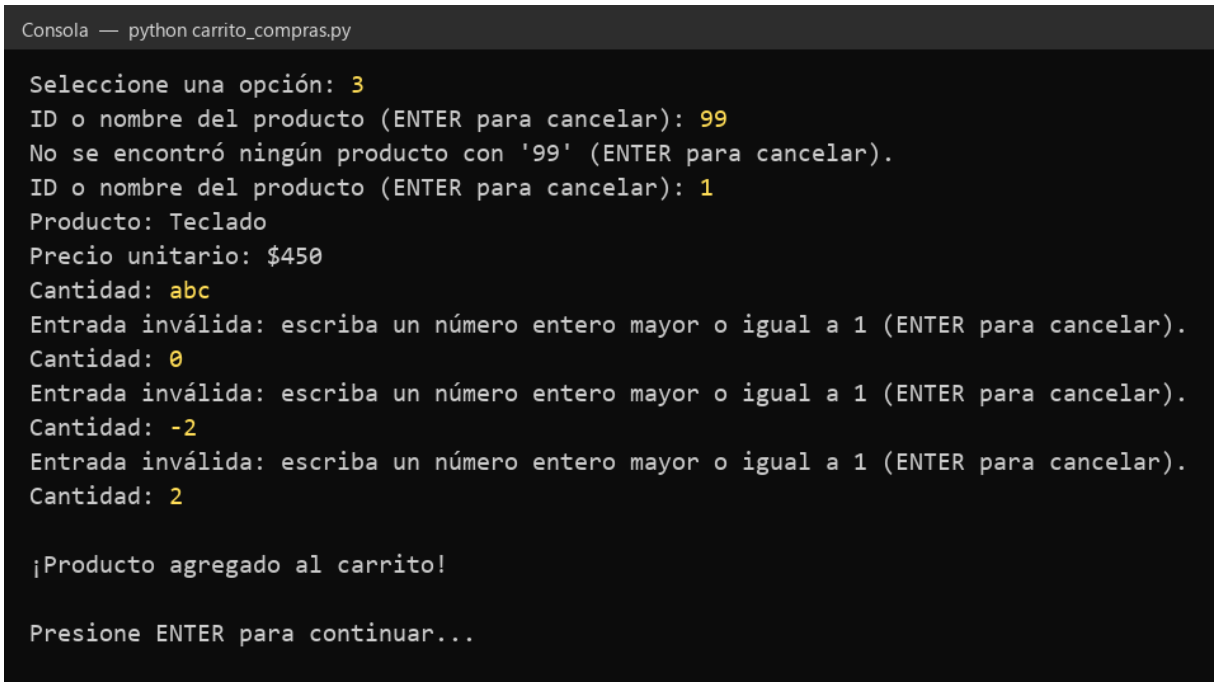


Figura 14. Validación: producto inexistente y cantidades inválidas

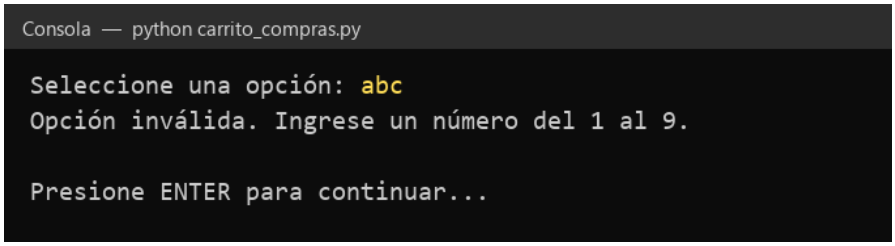


Figura 15. Validación: opción de menú inválida

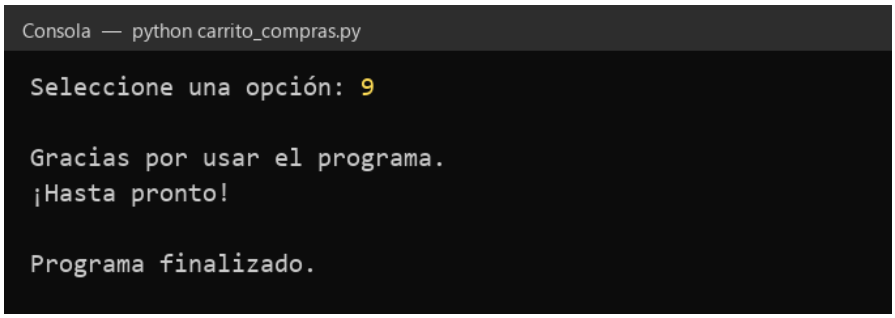


Figura 16. Opción 9: salir del programa

Conclusiones

El programa cumple los nueve requerimientos funcionales (R1 a R9) y los cinco requerimientos no funcionales: interfaz de consola, validación de entradas, manejo de errores, orientación a objetos y almacenamiento en listas. El uso de propiedades calculadas (`subtotal`, `total`) garantiza que los importes se actualicen automáticamente cada vez que cambia el carrito, y la excepción `CarritoError` separa las reglas de negocio de la interfaz. Por diseño, los datos se guardan solo en memoria, por lo que se pierden al cerrar el programa.

Anexo A. Código fuente completo (carrito_compras.py)

```
"""Carrito de compras (consola) - Proyecto de listas.

Simula un carrito de compras en una tienda. Toda la información vive en
memoria, dentro de listas:

    Catalogo.productos  -> lista de objetos Producto
    Carrito.items       -> lista de objetos ItemCarrito

Ejecución:  python carrito_compras.py
"""

import os
import sys
import unicodedata


# ---------------------------------------------------------------------------
# Utilidades
# ---------------------------------------------------------------------------
class CarritoError(Exception):
    """Error de negocio (producto inexistente, cantidad inválida, etc.)."""


def normalizar(texto):
    """Minúsculas y sin acentos, para comparar nombres ('Audífonos' == 'audifonos')."""
    sin_acentos = unicodedata.normalize("NFD", texto)
    sin_acentos = "".join(c for c in sin_acentos if not unicodedata.combining(c))
    return sin_acentos.casefold().strip()


def formatear_dinero(monto):
    """450 -> '$450'; 450.5 -> '$450.50'."""
    if float(monto).is_integer():
        return f"${int(monto)}"
    return f"${monto:.2f}"


def construir_tabla(encabezados, filas, alineacion):
    """Devuelve la tabla como lista de líneas de texto.

    alineacion: una letra por columna, 'i' (izquierda) o 'd' (derecha).
    """
    filas_txt = [[str(celda) for celda in fila] for fila in filas]
    anchos = [len(h) for h in encabezados]
    for fila in filas_txt:
        for i, celda in enumerate(fila):
            anchos[i] = max(anchos[i], len(celda))

    def formatear(fila):
        celdas = [
            celda.ljust(anchos[i]) if alineacion[i] == "i" else celda.rjust(anchos[i])
            for i, celda in enumerate(fila)
        ]
        return " | ".join(celdas)

    encabezado = formatear(encabezados)
    linea = "-" * len(encabezado)
    return [linea, encabezado, linea] + [formatear(f) for f in filas_txt] + [linea]


def filtrar_por_referencia(elementos, referencia):
    """Busca en una lista de elementos con .id y .nombre.
```

```
    Si la referencia es un número se busca por ID; si no, por nombre
    (coincidencia exacta primero y, si no hay, coincidencia parcial).
    """
    ref = referencia.strip()
    if not ref:
        return []
    if ref.isascii() and ref.isdigit():
        return [e for e in elementos if e.id == int(ref)]
    ref_norm = normalizar(ref)
    exactos = [e for e in elementos if normalizar(e.nombre) == ref_norm]
    if exactos:
        return exactos
    return [e for e in elementos if ref_norm in normalizar(e.nombre)]


# ---------------------------------------------------------------------------
# Modelo
# ---------------------------------------------------------------------------
class Producto:
    """Producto del catálogo: ID, nombre, categoría y precio."""

    def __init__(self, id_producto, nombre, categoria, precio):
        if precio < 0:
            raise CarritoError("El precio de un producto no puede ser negativo.")
        self.id = id_producto
        self.nombre = nombre
        self.categoria = categoria
        self.precio = precio

    def como_lista(self):
        return [self.id, self.nombre, self.precio]


class ItemCarrito:
    """Línea del carrito: un producto y la cantidad que se desea comprar."""

    def __init__(self, producto, cantidad):
        self.producto = producto
        self.cantidad = cantidad

    @property
    def id(self):
        return self.producto.id

    @property
    def nombre(self):
        return self.producto.nombre

    @property
    def precio(self):
        return self.producto.precio

    @property
    def cantidad(self):
        return self._cantidad

    @cantidad.setter
    def cantidad(self, valor):
        if not isinstance(valor, int) or valor < 1:
            raise CarritoError("La cantidad debe ser un número entero mayor o igual a 1.")
        self._cantidad = valor

    @property
    def subtotal(self):
```

```
        return self.precio * self.cantidad

    def como_lista(self):
        return [self.id, self.nombre, self.precio, self.cantidad]


class Catalogo:
    """Catálogo de productos, almacenado en una lista."""

    def __init__(self, productos=None):
        self.productos = list(productos) if productos else []

    def agregar(self, producto):
        if self.buscar_por_id(producto.id) is not None:
            raise CarritoError(f"Ya existe un producto con ID {producto.id}.")
        self.productos.append(producto)

    def buscar_por_id(self, id_producto):
        for producto in self.productos:
            if producto.id == id_producto:
                return producto
        return None

    def buscar_por_nombre(self, texto):
        """Productos cuyo nombre contiene el texto (sin distinguir acentos ni mayúsculas)."""
        texto_norm = normalizar(texto)
        if not texto_norm:
            return []
        return [p for p in self.productos if texto_norm in normalizar(p.nombre)]


class Carrito:
    """Carrito de compras, almacenado en una lista de ItemCarrito."""

    def __init__(self):
        self.items = []

    def esta_vacio(self):
        return len(self.items) == 0

    def buscar_item(self, id_producto):
        for item in self.items:
            if item.id == id_producto:
                return item
        return None

    def agregar(self, producto, cantidad):
        """Agrega el producto; si ya estaba en el carrito suma la cantidad."""
        item = self.buscar_item(producto.id)
        if item is None:
            item = ItemCarrito(producto, cantidad)
            self.items.append(item)
        else:
            item.cantidad = item.cantidad + cantidad
        return item

    def modificar_cantidad(self, id_producto, nueva_cantidad):
        item = self.buscar_item(id_producto)
        if item is None:
            raise CarritoError("Ese producto no está en el carrito.")
        item.cantidad = nueva_cantidad
        return item

    def eliminar(self, id_producto):
        item = self.buscar_item(id_producto)
```

```
        if item is None:
            raise CarritoError("Ese producto no está en el carrito.")
        self.items.remove(item)
        return item

    def vaciar(self):
        self.items.clear()

    @property
    def total(self):
        return sum(item.subtotal for item in self.items)

    @property
    def total_articulos(self):
        return sum(item.cantidad for item in self.items)

    @property
    def total_productos(self):
        return len(self.items)


def catalogo_inicial():
    """Catálogo precargado de la tienda."""
    return Catalogo([
        Producto(1, "Teclado", "Accesorios", 450),
        Producto(2, "Mouse", "Accesorios", 250),
        Producto(3, "Monitor", "Electrónica", 3200),
        Producto(4, "Audífonos", "Audio", 800),
        Producto(5, "Memoria USB", "Almacenamiento", 180),
    ])


# ---------------------------------------------------------------------------
# Interfaz de consola
# ---------------------------------------------------------------------------
class AplicacionCarrito:
    """Menú de consola que conecta al usuario con el catálogo y el carrito."""

    OPCIONES_MENU = [
        "Mostrar catálogo",
        "Buscar producto",
        "Agregar producto al carrito",
        "Ver carrito",
        "Modificar cantidad",
        "Eliminar producto",
        "Vaciar carrito",
        "Finalizar compra",
        "Salir",
    ]

    def __init__(self):
        self.catalogo = catalogo_inicial()
        self.carrito = Carrito()

    # ----- entrada / salida -----
    @staticmethod
    def limpiar_pantalla():
        if sys.stdout.isatty():
            os.system("cls" if os.name == "nt" else "clear")

    @staticmethod
    def pausar(mensaje="Presione ENTER para continuar..."):
        print()
        input(mensaje)
```

```
    @staticmethod
    def leer_entero_positivo(mensaje):
        """Pide un entero >= 1. Devuelve None si el usuario deja la entrada vacía."""
        while True:
            texto = input(mensaje).strip()
            if not texto:
                return None
            if texto.isascii() and texto.isdigit() and int(texto) >= 1:
                return int(texto)
            print("Entrada inválida: escriba un número entero mayor o igual a 1 "
                  "(ENTER para cancelar).")

    @staticmethod
    def confirmar(mensaje):
        """Pregunta (s/n) hasta recibir una respuesta válida."""
        while True:
            respuesta = normalizar(input(f"{mensaje} (s/n): "))
            if respuesta in ("s", "si"):
                return True
            if respuesta in ("n", "no"):
                return False
            print("Responda 's' para sí o 'n' para no.")

    @staticmethod
    def titulo(texto, ancho=0):
        print()
        print(f"=== {texto} ===".center(ancho))

    def elegir(self, elementos, mensaje):
        """Pide un ID o nombre y devuelve el elemento elegido (None si se cancela)."""
        while True:
            referencia = input(mensaje).strip()
            if not referencia:
                return None
            coincidencias = filtrar_por_referencia(elementos, referencia)
            if len(coincidencias) == 1:
                return coincidencias[0]
            if not coincidencias:
                print(f"No se encontró ningún producto con '{referencia}' "
                      "(ENTER para cancelar).")
            else:
                print("Hay varias coincidencias, indique el ID del producto:")
                self.imprimir_lista_corta(coincidencias)

    @staticmethod
    def imprimir_lista_corta(elementos):
        for linea in construir_tabla(
            ["ID", "Producto"], [[e.id, e.nombre] for e in elementos], "id"
        ):
            print(linea)

    # ----- pantallas -----
    def imprimir_catalogo(self, productos, titulo="CATÁLOGO DE PRODUCTOS"):
        filas = [[p.id, p.nombre, p.categoria, formatear_dinero(p.precio)] for p in productos]
        tabla = construir_tabla(["ID", "Producto", "Categoría", "Precio"], filas, "iiid")
        self.titulo(titulo, len(tabla[0]))
        for linea in tabla:
            print(linea)

    def imprimir_carrito(self, titulo="CARRITO DE COMPRAS", con_precio=True):
        if con_precio:
            encabezados = ["ID", "Producto", "Precio", "Cant.", "Subtotal"]
            filas = [[i.id, i.nombre, formatear_dinero(i.precio), i.cantidad,
                      formatear_dinero(i.subtotal)] for i in self.carrito.items]
            alineacion = "iiddd"
```

```
        else:
            encabezados = ["ID", "Producto", "Cant.", "Subtotal"]
            filas = [[i.id, i.nombre, i.cantidad, formatear_dinero(i.subtotal)]
                     for i in self.carrito.items]
            alineacion = "iidd"
        tabla = construir_tabla(encabezados, filas, alineacion)
        self.titulo(titulo, len(tabla[0]))
        for linea in tabla:
            print(linea)
        return len(tabla[0])

    def carrito_vacio(self):
        """Avisa si el carrito está vacío. Devuelve True en ese caso."""
        if self.carrito.esta_vacio():
            print("\nEl carrito está vacío.")
            self.pausar()
            return True
        return False

    # ----- opciones del menú -----
    def mostrar_menu(self):
        self.limpiar_pantalla()
        self.titulo("CARRITO DE COMPRAS")
        print()
        for numero, texto in enumerate(self.OPCIONES_MENU, start=1):
            print(f"{numero}. {texto}")
        print()

    def opcion_mostrar_catalogo(self):
        self.imprimir_catalogo(self.catalogo.productos)
        self.pausar("Presione ENTER para volver al menú...")

    def opcion_buscar(self):
        print("\nBuscar producto")
        print("1. Por ID")
        print("2. Por nombre")
        modo = input("Seleccione una opción (ENTER para cancelar): ").strip()
        if modo == "1":
            id_producto = self.leer_entero_positivo("ID del producto: ")
            if id_producto is None:
                return
            producto = self.catalogo.buscar_por_id(id_producto)
            resultados = [producto] if producto else []
        elif modo == "2":
            texto = input("Nombre del producto: ").strip()
            if not texto:
                return
            resultados = self.catalogo.buscar_por_nombre(texto)
        elif modo == "":
            return
        else:
            print("Opción inválida. Elija 1 o 2.")
            self.pausar()
            return

        if resultados:
            self.imprimir_catalogo(resultados, "RESULTADOS DE LA BÚSQUEDA")
        else:
            print("\nNo se encontraron productos.")
        self.pausar()

    def opcion_agregar(self):
        producto = self.elegir(self.catalogo.productos,
                               "ID o nombre del producto (ENTER para cancelar): ")
        if producto is None:
```

```
            return
        print(f"Producto: {producto.nombre}")
        print(f"Precio unitario: {formatear_dinero(producto.precio)}")
        cantidad = self.leer_entero_positivo("Cantidad: ")
        if cantidad is None:
            return
        item = self.carrito.agregar(producto, cantidad)
        print("\n¡Producto agregado al carrito!")
        if item.cantidad != cantidad:
            print(f"(Ya estaba en el carrito: ahora lleva {item.cantidad} unidades.)")
        self.pausar()

    def opcion_ver_carrito(self):
        if self.carrito_vacio():
            return
        ancho = self.imprimir_carrito()
        print(f"TOTAL: {formatear_dinero(self.carrito.total)}".rjust(ancho))
        self.pausar("Presione ENTER para volver al menú...")

    def opcion_modificar(self):
        if self.carrito_vacio():
            return
        item = self.elegir(self.carrito.items,
                           "ID o nombre del producto a modificar (ENTER para cancelar): ")
        if item is None:
            return
        print(f"Producto: {item.nombre}")
        print(f"Cantidad actual: {item.cantidad}")
        nueva = self.leer_entero_positivo("Nueva cantidad: ")
        if nueva is None:
            return
        self.carrito.modificar_cantidad(item.id, nueva)
        print("\n¡Cantidad actualizada!")
        print(f"Nuevo subtotal: {formatear_dinero(item.subtotal)}")
        print(f"Total del carrito: {formatear_dinero(self.carrito.total)}")
        self.pausar()

    def opcion_eliminar(self):
        if self.carrito_vacio():
            return
        item = self.elegir(self.carrito.items,
                           "ID o nombre del producto a eliminar (ENTER para cancelar): ")
        if item is None:
            return
        print(f"Producto: {item.nombre}")
        if self.confirmar("¿Está seguro que desea eliminar este producto?"):
            self.carrito.eliminar(item.id)
            print("\n¡Producto eliminado del carrito!")
        else:
            print("\nOperación cancelada.")
        self.pausar()

    def opcion_vaciar(self):
        if self.carrito_vacio():
            return
        if self.confirmar("¿Está seguro que desea vaciar el carrito?"):
            self.carrito.vaciar()
            print("\n¡Carrito vaciado!")
        else:
            print("\nOperación cancelada.")
        self.pausar()

    def opcion_finalizar(self):
        if self.carrito_vacio():
            return
```

```
        self.imprimir_carrito("RESUMEN DE COMPRA", con_precio=False)
        print()
        print(f"Productos:     {self.carrito.total_productos}")
        print(f"Artículos:     {self.carrito.total_articulos}")
        print(f"Total a pagar: {formatear_dinero(self.carrito.total)}")
        print("\nGracias por su compra.")
        self.carrito.vaciar()  # la compra termina y el carrito se reinicia
        self.pausar("Presione ENTER para volver al menú...")

    # ----- ciclo principal -----
    def ejecutar(self):
        acciones = {
            "1": self.opcion_mostrar_catalogo,
            "2": self.opcion_buscar,
            "3": self.opcion_agregar,
            "4": self.opcion_ver_carrito,
            "5": self.opcion_modificar,
            "6": self.opcion_eliminar,
            "7": self.opcion_vaciar,
            "8": self.opcion_finalizar,
        }
        try:
            while True:
                self.mostrar_menu()
                opcion = input("Seleccione una opción: ").strip()
                if opcion == "9":
                    break
                accion = acciones.get(opcion)
                if accion is None:
                    print("Opción inválida. Ingrese un número del 1 al 9.")
                    self.pausar()
                    continue
                try:
                    accion()
                except CarritoError as error:
                    print(f"\nError: {error}")
                    self.pausar()
        except (KeyboardInterrupt, EOFError):
            print()
        print("\nGracias por usar el programa.")
        print("¡Hasta pronto!")
        print("\nPrograma finalizado.")


def main():
    # Evita que un carácter no soportado por la consola detenga el programa.
    for flujo in (sys.stdout, sys.stderr):
        try:
            flujo.reconfigure(errors="replace")
        except (AttributeError, ValueError):
            pass
    AplicacionCarrito().ejecutar()


if __name__ == "__main__":
    main()
```